

PATCH 4

Detalhe, Imersão e Personalização

Mini-Jogo de Basquetebol 3D — ICG 2025/2026

Este patch foca-se em polimento visual, imersão e personalização. Inclui correções de jogabilidade, enriquecimento do cenário exterior, melhorias ao personagem e um menu de customização interativo.

Índice do Patch 4

#	Melhoria	Ficheiro(s)	Dificuldade
4.1	Precisão no perfect corrigida	shot.js	Baixa
4.2	Árvores realistas com movimento de vento	world/park.js	Média
4.3	Pássaros a atravessar o parque	world/park.js	Média
4.4	Pessoas no parque (4 cenas)	world/npcs.js (novo)	Alta
4.5	Linha dos 3 pontos completa	world/court.js	Baixa
4.6	Salto ao lançar	world/player.js, main.js	Média
4.7	Câmara 1ª pessoa fixa no cesto ao lançar	game/camera.js, main.js	Média
4.8	Equipamento NBA style verde/preto	world/player.js	Baixa
4.9	Remover shadow blob dos pés	world/player.js	Baixa
4.10	Sapatilhas no jogador	world/player.js	Baixa
4.11	Menu de customização (roupa, bola, número)	ui/menu.js, ui.css (novo)	Alta

4.1 — Precisão no Perfect Corrigida

Mesmo acertando na zona verde a bola não entra consistentemente porque o código atual ainda tem resquícios do sistema antigo — um random na velocidade e o erro horizontal não é verdadeiramente zero no verde. Com o sistema de precisão pura planeado, o verde deve dar erro = 0 e velocidade = idealSpeed exata.

Código atual com problemas em shot.js:

```
// Problema 1: random na velocidade mesmo no perfect speed = idealSpeed * (1 +  
randomBetween(-0.01, 0.01)); // Problema 2: erro horizontal calculado mesmo no perfect  
const missFactor = clamp(Math.abs(power - center) / range, 0, 1);
```

Código corrigido:

```
if (isPerfect) { speed = idealSpeed; // sem random – velocidade exata horizontalError = 0;  
// sem desvio – tiro perfeito garantido } else { speed = idealSpeed; // velocidade sempre  
ideal (precisão pura) const center = (tuning.perfectMin + tuning.perfectMax) * 0.5; const  
missFactor = clamp(Math.abs(power - center) / center, 0, 1); horizontalError =  
degToRad(tuning.maxHorizontalErrorDeg) * missFactor * (Math.random() < 0.5 ? -1 : 1); }
```

4.2 — Árvores Realistas com Movimento de Vento

As árvores atuais usam uma esfera única para a copa. Serão substituídas por árvores procedurais com múltiplas camadas de esferas em tons de verde diferentes e movimento suave de vento sincronizado com o tempo de jogo.

Estrutura de cada árvore:

Parte	Geometria	Detalhe
Tronco	CylinderGeometry(0.18, 0.28, 3.5, 8)	Ligeiramente cônico, castanho escuro (#4A2F0A), castShadow
Copa principal	SphereGeometry(1.8, 10, 10)	Verde escuro (#1B5E20), posição y=3.8
Copa secundária	SphereGeometry(1.4, 8, 8)	Verde médio (#2E7D32), posição y=4.8, offset lateral ±0.4
Copa terciária	SphereGeometry(1.0, 8, 8)	Verde claro (#388E3C), posição y=5.6, offset lateral diferente
Ramos (2–3)	CylinderGeometry(0.04, 0.07, 1.2, 6)	mesma cor do tronco, inclinados 30–45°, saem do tronco a y

Movimento de vento:

Cada copa tem um fator de fase aleatório atribuído na criação para que as árvores não balancem todas em sincronia. O movimento é aplicado em cada frame com um seno muito suave no eixo X e Z da copa.

Em `update(delta)` no `park.js`:

```
trees.forEach(tree => { const sway = Math.sin(time * 0.8 + tree.phase) * 0.03;
tree.topGroup.rotation.x = sway; tree.topGroup.rotation.z = sway * 0.6; });
```

4.3 — Pássaros a Atravessar o Parque

Grupos de 2-3 pássaros aparecem aleatoriamente de um lado do parque, atravessam voando e desaparecem do outro lado. Cada pássaro tem animação de bater de asas. Surgem a alturas e velocidades ligeiramente diferentes para parecer natural.

Aspeto	Detalhe
Geometria das asas	Duas PlaneGeometry(0.6, 0.25) em V — asa esquerda e direita
Corpo	SphereGeometry(0.1) achatada no centro do V
Cor	MeshBasicMaterial preto/cinza escuro (#222222)
Animação das asas	rotation.z de cada asa oscila com $\text{Math.sin}(\text{time} * 6 + \text{fase}) * 0.5$
Velocidade de voo	Entre 4 e 7 unidades/s, ligeiramente diferente por pássaro
Altitude	Entre y=12 e y=22, aleatório por grupo
Spawn	A cada 8–15s (intervalo aleatório), aparecem em x=-50 ou x=+50
Despawn	Quando saem da cena (x > 55 ou x < -55) são reposicionados

4.4 — Pessoas no Parque (4 Cenas)

NPCs simples colocados estrategicamente no parque para dar vida ao cenário. Usam a mesma estrutura de geometrias básicas do jogador principal mas com escalas, cores e poses diferentes. Serão criados em `src/world/npcs.js`.

As 4 cenas:

Cena	Personagens	Comportamento	Posição
Família a passear	2 adultos + 1 criança	Escala 0.6, suave no parque, animação de acordo com o ritmo	À esquerda dos 3 pontos
Pessoa sentada a ler	1 adulto sentado	Pose estática: torso inclinado 20° para frente, braços à relaxe segurando o livro	Debaixo de uma árvore à direita
Casal a passear	2 adultos	Loop linear lento de um extremo ao outro da área, de forma sincronizada	Às costas da área, de forma sincronizada
Pessoa parada	1 adulto	Pose idle — ligeiro balanço do torso	Debaixo de uma árvore

Estrutura técnica dos NPCs:

Cada NPC é um objeto com: grupo Three.js, tipo de comportamento, path de movimento (array de Vector3 para os que andam), velocidade e um animTime próprio. A função updateNPCs(delta) em main.js itera todos os NPCs e aplica o movimento e animação correspondente.

Pseudocódigo em npcs.js:

```
export function createNPCs(scene) { const npcs = []; // Família const family = createFamilyGroup(scene); npcs.push({ group: family, type: 'loop', path: familyPath, speed: 1.2, progress: 0, animTime: 0 }); // Pessoa a ler const reader = createNPC(scene, { scale: 1, shirtColor: 0xE53935 }); applyReadingPose(reader); npcs.push({ group: reader, type: 'static' }); // ... return npcs; }
```

4.5 — Linha dos 3 Pontos Completa

A linha dos 3 pontos atual está incompleta — o arco é desenhado corretamente mas as linhas retas laterais que ligam o arco ao baseline têm um erro no cálculo de arcDelta que causa cortes em certas zonas do mapeamento da textura.

Problema em createLinesTexture() no court.js:

```
// Cálculo atual – arcDelta pode dar valores negativos em certas configurações: const arcDelta = Math.sqrt(Math.max(0.001, COURT.threePointRadius ** 2 - sideLineX ** 2)); const arcTouchZ = rim.z - sign * arcDelta; // As linhas laterais deviam ir do baseline ATÉ ao ponto de tangência do arco. // O problema: arcTouchZ é calculado em coordenadas de mundo mas mapY // converte para píxeis de textura – o sinal pode inverter conforme o lado.
```

Correção:

```
// Calcular corretamente o ponto de tangência para cada lado: const rimZ = getRimCenter(sign).z; const arcTouchZ = rimZ - sign * Math.sqrt( Math.max(0, COURT.threePointRadius ** 2 - sideLineX ** 2) ); // Garantir que as linhas laterais vão SEMPRE do baseline ao arcTouchZ: const lineStart = mapY(sign * COURT.halfLength); // baseline const lineEnd = mapY(arcTouchZ); // tangência do arco ctx.beginPath(); ctx.moveTo(mapX(-sideLineX), Math.min(lineStart, lineEnd)); ctx.lineTo(mapX(-sideLineX), Math.max(lineStart, lineEnd)); ctx.stroke(); // repetir para o lado oposto (+sideLineX)
```

4.6 — Salto ao Lançar

Durante a preparação do lançamento o jogador sobe ligeiramente do chão, sincronizado com a elevação dos braços. O pico do salto coincide com o momento de release da bola, e o jogador pausa suavemente depois.

Fase	Timing	Offset Y
Preparação (braços sobem)	0 → releaseTime (0.2s)	lerp de 0 até +0.35
Release (bola sai)	releaseTime	pico em +0.35
Aterragem (braços descem)	releaseTime → duration (0.64s)	lerp de +0.35 até 0

Implementação em updateShootSequence() no main.js:

```
// Durante a preparação: const jumpOffset = prep * 0.35; // prep vai de 0 a 1 até ao release
player.setJumpOffset(jumpOffset); // Após o release: const coolDown = (shootSequence.timer
- shootSequence.releaseTime) / (shootSequence.duration - shootSequence.releaseTime);
player.setJumpOffset((1 - coolDown) * 0.35); // desce suavemente
```

4.7 — Câmara 1ª Pessoa Fixa no Cesto ao Lançar

Atualmente em 1ª pessoa ao lançar a câmara vira para o lado oposto ao cesto. O objetivo é que ao iniciar o lançamento a câmara faça lock-on ao cesto e fique a olhar para ele durante todo o lançamento, permitindo ver a bola entrar.

Fase	Duração	Comportamento da câmara
Trigger (clique)	Instantâneo	Calcular ângulo para o cesto mais próximo. Bloquear rato.
Lock-On	~0.35s	fpYaw e fpPitch interpolam suavemente para encarar o cesto. Jogador roda i
Charge (segurar)	Enquanto segurarmos	Câmara fixa a olhar para o cesto. Barra de timing visível.
Release + voo da bola	Até bola pousar no cesto	Câmara mantém-se a olhar para o cesto. Rato continua bloqueado.
Fim	Após score ou ratão desbloqueado	Rato desbloqueado. Jogador retoma controlo livre da câmara.

Correção do ângulo em camera.js:

```
// PROBLEMA: atan2 devolvia o ângulo invertido – sinal trocado no Z // ERRADO: const
targetYaw = Math.atan2(dx, dz); // CORRETO: vetor da cabeça do jogador PARA o cesto const
toHoop = new THREE.Vector3( hoop.rimCenter.x - headPos.x, 0, hoop.rimCenter.z - headPos.z
); const targetYaw = Math.atan2(toHoop.x, toHoop.z); const targetPitch = -Math.atan2(
hoop.rimCenter.y - headPos.y, toHoop.length() ); // pitch negativo = olhar para cima
```

4.8 — Equipamento NBA Style Verde/Preto

O equipamento atual usa cores genéricas. Será substituído por um conjunto inspirado no estilo Boston Celtics com verde escuro, preto e detalhes brancos.

Parte	Geometria	Material	Cor
Camisola (torso)	BoxGeometry(0.72, 0.95, 0.42)	StandardMaterial	Verde escuro #1B5E20
Faixa lateral	BoxGeometry(0.05, 0.95, 0.42) * 2	StandardMaterial	Preto #111111
Número (peito/costas)	BoxGeometry(0.3, 0.35, 0.15)	Textura canvas com número	Branco #FFFFFF sobre verde
Calções	BoxGeometry(0.62, 0.52, 0.36)	StandardMaterial	Preto #111111
Meias (parte alta)	CylinderGeometry — parte superior	StandardMaterial	Verde #1B5E20 com faixa branca
Meias (parte baixa)	CylinderGeometry — parte inferior	StandardMaterial	Branco #F5F5F5
Pele (cabeça/braços)	SphereGeometry + CylinderGeometry	StandardMaterial	Mantém-se #F3D4B8

4.9 — Remover Shadow Blob dos Pés

O shadowBlob é um CircleGeometry preto semi-transparente debaixo do jogador que simula uma sombra falsa. Com o sistema de sombras reais já ativo (castShadow / receiveShadow) este elemento é redundante e visualmente inconsistente com o novo cenário exterior.

Remover em player.js:

```
// REMOVER – criação do shadowBlob: // const shadowBlob = new THREE.Mesh(...); //
scene.add(shadowBlob); // REMOVER – referências em setPositionAndYaw e setJumpOffset: //
```

```
shadowBlob.position.set(...); // shadowBlob.material.opacity = ...;
```

4.10 — Sapatilhas no Jogador

Atualmente o jogador não tem pés — as pernas terminam abruptamente. Serão adicionadas duas sapatilhas como BoxGeometry achatadas e ligeiramente mais largas que as pernas, presas a cada legPivot para acompanhar a animação.

Parte	Geometria	Cor	Posição
Parte superior da sapatilha	BoxGeometry(0.22, 0.14, 0.06)	Branco #111111	Fundo da perna, y=-0.85 relativo à anca
Sola	BoxGeometry(0.24, 0.06, 0.01)	Branco #EEEEEE	Abaixo da parte superior, y=-0.95
Detalhe lateral	BoxGeometry(0.23, 0.06, 0.01)	Verde #1B5E20	Faixa no meio da sapatilha

As sapatilhas são adicionadas como filhas do legPivot de cada perna, garantindo que acompanham automaticamente todas as animações sem código extra.

4.11 — Menu de Customização

Um ícone de camisola no HUD abre um painel lateral de customização. O jogador pode mudar as cores da camisola, calções, bola e o número em tempo real. As alterações aplicam-se instantaneamente nos materiais Three.js sem recarregar.

Interface do menu:

Secção	Controlo	Implementação
Cor da camisola	Palette com 8 círculos coloridos clicáveis	Do dataset jerseyMaterial.color.set(hexColor)
Cor dos calções	Palette com 8 círculos coloridos clicáveis	Do dataset shortsMaterial.color.set(hexColor)
Cor da bola	Palette com 6 círculos (laranja, verde, amarelo, azul, vermelho, branco)	Aplicar: regenera a esfera da bola com nova cor base
Número (1–99)	Botões — e + com display do número atual	Atualizar: regenerar textura canvas com novo número

Estrutura HTML do menu em index.html:

```
div#customize-btn → icone de camisola no HUD (canto superior direito)
div#customize-panel.hidden → painel lateral h2: "Personalizacao"
div.custom-section → label "Camisola" + div.color-palette#jersey-colors
div.custom-section → label "Calcoes" + div.color-palette#shorts-colors
div.custom-section → label "Bola" + div.color-palette#ball-colors
div.custom-section → label "Numero"
div.number-picker:
  button#num-down [-] + span#num-display [23] + button#num-up [+]
```

Ligação ao Three.js em ui.js:

```
// Expor materiais e função de update da bola no createHUD:
export function createHUD(jerseyMat, shortsMat, updateBallTexture) {
  // ... jerseyColors.forEach(swatch => {
  swatch.addEventListener('click', () => {
    jerseyMat.color.set(swatch.dataset.color);
  });
  numberUp.addEventListener('click', () => {
    currentNumber = Math.min(99, currentNumber + 1);
    numDisplay.textContent = currentNumber;
    updateJerseyNumber(currentNumber);
    // regenera textura canvas
  });
}
```

Ordem de Implementação do Patch 4

As melhorias mais simples e isoladas primeiro para validar rapidamente, deixando as mais complexas e interdependentes para o final.

Ordem	Melhoria	Razão
1º	4.9 — Remover shadow blob	1 linha de código, resultado imediato
2º	4.1 — Precisão no perfect	Isolado em shot.js, testar todos os lançamentos
3º	4.5 — Linha dos 3 pontos	Isolado em court.js, verificar visualmente
4º	4.6 — Salto ao lançar	Depende do sistema de shoot existente — fácil de adicionar
5º	4.8 — Equipamento verde/preto	Isolado em player.js, substituir materiais
6º	4.10 — Sapatilhas	Continuação do ponto anterior, mesmo ficheiro
7º	4.7 — Câmara 1ª pessoa no cesto	Corrigir sinal do ângulo em camera.js
8º	4.2 — Árvores realistas + vento	Substituir geometria das árvores em park.js
9º	4.3 — Pássaros	Novo sistema em park.js, testar spawning e animação
10º	4.4 — NPCs no parque	Criar npcs.js do zero — o mais trabalhoso do patch
11º	4.11 — Menu de customização	Depende dos materiais finais do jogador e bola

Ferramentas de IA integradas no desenvolvimento e apoio à implementação: OpenAI ChatGPT · Anthropic Claude